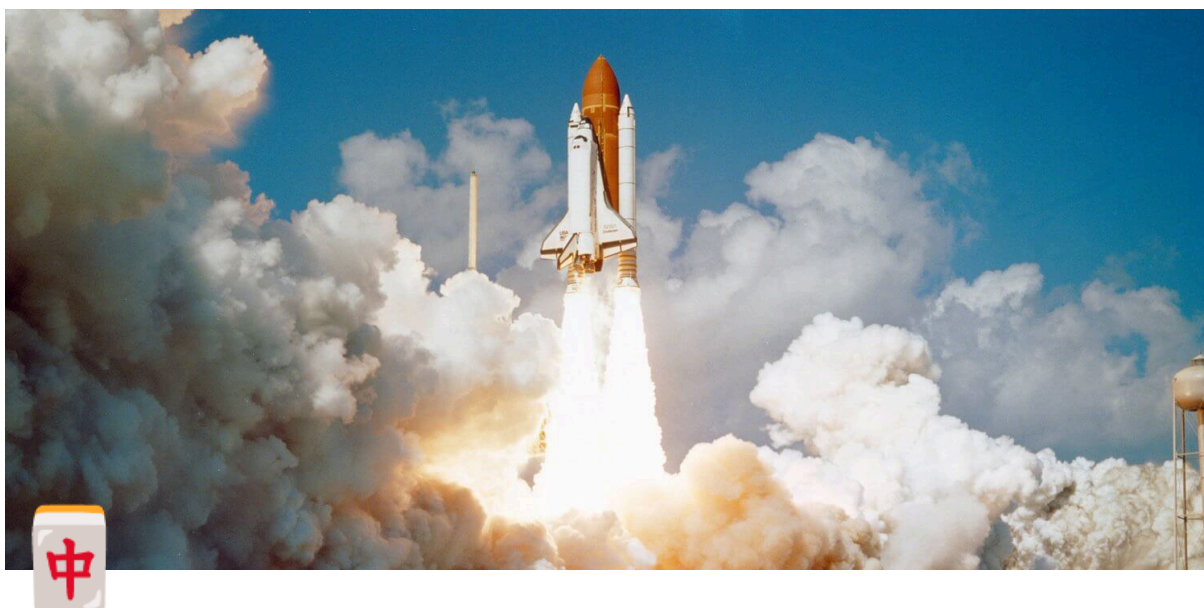




Guía de ejercicios 1.3.2

Guía de Ejercicios: Programación Orientada a Objetos en Java

Parte I — Ejercicios para desarrollar en clase

Ejercicio 1 — Gestión de productos bancarios

Un banco necesita representar dos tipos de productos: **Cuenta Corriente** y **Cuenta de Ahorro**.

Ambas comparten información general, pero aplican reglas diferentes para administrar y retirar fondos.

Requerimientos

Diseña una clase abstracta llamada `ProductoBancario` con:

- número de producto;
- nombre del titular;
- saldo.

Todos los atributos deben estar encapsulados.

Incluya un constructor y el método:

```
public void mostrarResumen()
```

También implemente:

```
public void depositar(double monto)
```

El método debe validar que el monto sea válido antes de modificar el saldo.

Declare además:

```
public abstract boolean girar(double monto);
```

Clase **CuentaCorriente**

Debe heredar de **ProductoBancario** e incorporar una **línea de crédito**.

La operación **girar(double monto)** debe permitir utilizar primero el saldo y, cuando sea necesario, la línea de crédito disponible.

Clase **CuentaAhorro**

Debe heredar de **ProductoBancario** e incorporar una **tasa de interés**.

No puede utilizar línea de crédito. Un giro solo puede realizarse si existe saldo suficiente.

Implemente además:

```
public void aplicarInteres()
```

Prueba

Desde **main**:

1. Cree una cuenta corriente y una cuenta de ahorro.
2. Realice depósitos.
3. Pruebe giros válidos e inválidos.
4. Aplique intereses a la cuenta de ahorro.

5. Muestre el resumen final.
6. Utilice referencias polimórficas:

```
ProductoBancario producto1 = new CuentaCorriente(...);  
ProductoBancario producto2 = new CuentaAhorro(...);
```

Conceptos

Herencia, abstracción, sobrescritura, `super`, métodos con parámetros, retorno `boolean` y polimorfismo.

Ejercicio 2 — Cálculo de prestaciones de salud

Una clínica necesita representar:

- consultas médicas;
- exámenes de laboratorio;
- sesiones de kinesiología.

Todas las prestaciones comparten información general, pero cada una calcula su valor final de manera diferente.

Clase abstracta `PrestacionSalud`

Debe contener:

- identificador;
- nombre del paciente;
- valor base.

Incluya un constructor y:

```
public void mostrarInformacion()
```

Declare:

```
public abstract double calcularValorFinal();
```

Clase `ConsultaMedica`

Debe incorporar:

- especialidad;
- porcentaje adicional.

El valor final debe considerar el valor base y el porcentaje adicional.

Clase **ExamenLaboratorio**

Debe incorporar:

- tipo de examen;
- condición de urgencia.

Si el examen es urgente, debe aplicar un recargo.

Clase **SesionKinesiologia**

Debe incorporar:

- duración en minutos;
- valor adicional por minuto.

El valor final debe considerar la duración.

Uso de **String**

Implemente:

```
public boolean coincideConPaciente(String texto)
```

La búsqueda debe ignorar diferencias entre mayúsculas y minúsculas y permitir verificar si el texto está contenido en el nombre del paciente.

Prueba

Cree una instancia de cada prestación y utilice referencias polimórficas:

```
PrestacionSalud prestacion1 = new ConsultaMedica(...);
PrestacionSalud prestacion2 = new ExamenLaboratorio(...);
PrestacionSalud prestacion3 = new SesionKinesiologia(...);
```

Ejecute `calcularValorFinal()` sobre las tres referencias.

Ejercicio 3 — Sistema de inversiones financieras

Una empresa permite invertir en:

- depósitos a plazo;
- fondos de inversión;
- acciones.

Cada instrumento calcula su rentabilidad y nivel de riesgo de forma diferente.

Clase abstracta **InstrumentoFinanciero**

Debe almacenar:

- código;
- nombre de la inversión;
- monto invertido.

Incluya:

```
public void mostrarResumen()
```

Declare:

```
public abstract double calcularRentabilidad();  
public abstract String obtenerNivelRiesgo();
```

Clase **DepositoPlazo**

Debe incorporar:

- tasa de interés;
- plazo en meses.

Su nivel de riesgo debe ser **"Bajo"**.

Clase **FondoInversion**

Debe incorporar:

- variación porcentual;
- comisión de administración.

La rentabilidad debe considerar ambas variables.

Clase **Accion**

Debe incorporar:

- precio de compra;
- precio actual;
- cantidad de acciones.

La rentabilidad debe considerar la variación del valor de las acciones.

Interfaz **Liquidable**

Declare:

```
double liquidar();
```

Los instrumentos que implementen esta interfaz deben calcular el monto correspondiente al liquidar la inversión. En este contexto, **liquidar una inversión significa cerrar la inversión y recuperar el dinero correspondiente a su valor actual.**

Depósito a plazo

Podría liquidarse recuperando:

```
Monto invertido + intereses generados
```

Fondo de inversión

Podría liquidarse recuperando:

```
Monto invertido + rentabilidad - comisión de liquidación
```

Acción

Podría liquidarse mediante la venta de las acciones al precio actual:

```
Precio actual × cantidad de acciones
```

Prueba

Para cada instrumento:

1. Muestre el resumen.
2. Calcule la rentabilidad.
3. Obtenga el nivel de riesgo.
4. Pruebe la liquidación cuando corresponda.

Demuestre polimorfismo mediante referencias de tipo `InstrumentoFinanciero`.

Ejercicio 4 — Gestión de pólizas de seguros

Debe almacenar los siguientes atributos:

- número de póliza;
- nombre del cliente;
- monto asegurado;
- fecha de inicio;
- cantidad de meses de vigencia.

Clase abstracta `PolizaSeguro`

Debe contener:

- número de póliza;
- nombre del cliente;
- monto asegurado.

Incluya:

```
public void mostrarInformacion()
```

Declare:

```
public abstract double calcularPrima();  
public abstract String obtenerTipoCobertura();
```

Clase `SeguroSalud`

Debe considerar:

- edad del asegurado;
- porcentaje de cobertura.

La prima debe calcularse considerando, como mínimo:

- el monto asegurado;
- la edad del asegurado.

El método:

```
obtenerTipoCobertura()
```

debe devolver una descripción correspondiente a la cobertura de salud.

Clase **SeguroVehículo**

Debe considerar:

- marca;
- año;
- valor comercial.

La prima debe calcularse considerando las características del vehículo.

El método:

```
obtenerTipoCobertura()
```

debe devolver una descripción correspondiente a la cobertura del vehículo.

Clase **SeguroVida**

Debe considerar:

- edad;
- duración del contrato.

La prima debe calcularse según las reglas definidas para este tipo de seguro.

El método:

```
obtenerTipoCobertura()
```

debe devolver una descripción correspondiente al seguro de vida.

Interfaz **Renovable**

Declare:

```
boolean renovar(int cantidadMeses);
```

Búsqueda del cliente utilizando **String**

La compañía necesita realizar una búsqueda simple de pólizas según el nombre del cliente.

Implemente en la clase **PolizaSeguro** el método:

```
public boolean coincideConCliente(String texto)
```

Este método debe recibir un texto de búsqueda como parámetro y determinar si dicho texto está contenido en el nombre del cliente asociado a la póliza.

La búsqueda no debe diferenciar entre mayúsculas y minúsculas.

Por ejemplo:

```
Nombre del cliente: "Juan Carlos Pérez"
```

```
Texto buscado: "juan"
```

```
Resultado: true
```

```
Nombre del cliente: "Juan Carlos Pérez"
```

```
Texto buscado: "María"
```

```
Resultado: false
```

Prueba

Cree una póliza de cada tipo y:

1. Muestre su información.

2. Calcule su prima.
 3. Muestre su cobertura.
 4. Pruebe la búsqueda por cliente.
 5. Pruebe la renovación cuando corresponda.
 6. Utilice referencias de tipo `PolizaSeguro`.
-

Parte II — Ejercicios integradores

Los siguientes ejercicios requieren analizar el problema antes de comenzar a programar.

No se entrega un modelo completo de clases. El estudiante debe determinar:

- qué clases son necesarias;
- qué responsabilidades tiene cada clase;
- qué atributos son comunes;
- dónde corresponde utilizar herencia;
- cuándo una interfaz representa una capacidad;
- qué métodos deben ser abstractos;
- dónde se aplica polimorfismo.

El sistema puede probarse mediante sentencias directas en `main` o mediante un menú simple.

Ejercicio integrador 1 — Módulo de operaciones bancarias

Un banco necesita desarrollar un módulo para administrar productos financieros.

Inicialmente debe considerar:

- cuentas corrientes;
- cuentas de ahorro;
- tarjetas de crédito.

No se deben utilizar colecciones. Los objetos necesarios pueden crearse directamente desde `main`.

Requerimiento 1 — Cliente

Diseñe una clase `Cliente` con:

- identificador;
- nombre completo;
- correo electrónico.

Implemente:

```
public boolean esCorreoValido()
```

Como mínimo, considere que un correo válido debe contener `@`.

Implemente también:

```
public boolean coincideConBusqueda(String texto)
```

La búsqueda debe ignorar diferencias entre mayúsculas y minúsculas.

Requerimiento 2 — Productos financieros

Analice las características comunes de los distintos productos y diseñe una abstracción adecuada.

Como mínimo, considere:

- número de producto;
- cliente asociado;
- estado.

Determine qué operaciones deben ser comunes y cuáles deben variar según el tipo de producto.

Puede considerar operaciones como:

```
depositar(double monto)
girar(double monto)
```

```
calcularMontoDisponible()
```

Requerimiento 3 — Cuenta Corriente

Debe considerar:

- saldo;
- línea de crédito;
- estado.

Debe permitir:

- depósitos;
- giros;
- consulta del monto disponible.

Requerimiento 4 — Cuenta de Ahorro

Debe considerar:

- saldo;
- tasa de interés;
- estado.

Debe permitir:

- depósitos;
- giros;
- aplicación de intereses;
- consulta de saldo.

No puede utilizar línea de crédito.

Requerimiento 5 — Tarjeta de Crédito

Debe considerar:

- cupo total;
- cupo utilizado;

- estado.

Debe permitir:

- realizar compras;
- consultar cupo disponible;
- registrar pagos.

Una compra no puede superar el cupo disponible.

Requerimiento 6 — Bloqueo

Analice si una interfaz representa adecuadamente la capacidad de bloquear un producto.

Defina cómo el bloqueo afecta las operaciones financieras.

Requerimiento 7 — Polimorfismo

Demuestre que los productos pueden ser tratados mediante una abstracción común:

```
ProductoFinanciero producto1 = new CuentaCorriente(...);  
ProductoFinanciero producto2 = new CuentaAhorro(...);  
ProductoFinanciero producto3 = new TarjetaCredito(...);
```

Opcionalmente puede utilizar:

```
ProductoFinanciero[] productos = new ProductoFinanciero[3];
```

Prueba mínima

El programa debe demostrar:

1. Creación de clientes.
2. Validación de correo.
3. Búsqueda por nombre.
4. Creación de productos.
5. Depósitos y giros.

6. Aplicación de intereses.
 7. Compras con tarjeta.
 8. Registro de pagos.
 9. Bloqueo, si corresponde al diseño.
 10. Consulta de saldos y cupos.
 11. Polimorfismo.
-

Ejercicio integrador 2 — Sistema de gestión de prestaciones de salud

Una institución necesita registrar y calcular prestaciones realizadas a pacientes.

El sistema debe considerar:

- consultas médicas;
- exámenes de laboratorio;
- sesiones de kinesiología.

Además, los pacientes pueden tener diferentes modalidades de cobertura.

Requerimiento 1 — Paciente

Diseñe una clase `Paciente` con:

- identificador;
- nombre completo;
- correo;
- tipo de cobertura.

Implemente:

```
public boolean esCorreoValido()
```

e:

```
public boolean coincideConBusqueda(String texto)
```

La búsqueda debe ignorar diferencias entre mayúsculas y minúsculas.

Requerimiento 2 — Prestaciones

Analice las características comunes y diseñe una clase abstracta

```
PrestacionSalud.
```

Como mínimo, considere:

- identificador;
- paciente;
- fecha;
- valor base.

Debe existir:

```
public abstract double calcularValorFinal();
```

También debe existir un comportamiento para mostrar el detalle de la prestación.

Requerimiento 3 — Consulta médica

Debe considerar:

- especialidad;
- profesional responsable;
- porcentaje adicional.

Requerimiento 4 — Examen de laboratorio

Debe considerar:

- tipo de examen;
- condición de urgencia.

Los exámenes urgentes deben aplicar una regla diferente.

Requerimiento 5 — Sesión de kinesiología

Debe considerar:

- tipo de tratamiento;
- duración;
- número de sesión.

Requerimiento 6 — Cobertura

Analice cómo representar diferentes modalidades, por ejemplo:

- atención particular;
- seguro de salud;
- convenio.

El diseño debe permitir calcular cuánto paga finalmente el paciente.

Puede utilizar una interfaz o una abstracción adicional.

Una operación posible es:

```
public double calcularCopago()
```

Requerimiento 7 — Polimorfismo

Debe ser posible trabajar con diferentes prestaciones mediante una abstracción común:

```
PrestacionSalud prestacion1 = new ConsultaMedica(...);  
PrestacionSalud prestacion2 = new ExamenLaboratorio(...);  
PrestacionSalud prestacion3 = new SesionKinesiologia(...);
```

El código principal debe poder ejecutar:

```
prestacion1.calcularValorFinal();  
prestacion2.calcularValorFinal();  
prestacion3.calcularValorFinal();
```

sin depender de la implementación concreta de cada prestación.

Prueba mínima

El programa debe demostrar:

1. Creación de pacientes.
 2. Validación de correos.
 3. Búsqueda por texto.
 4. Creación de prestaciones.
 5. Cálculo del valor final.
 6. Aplicación de cobertura o cálculo de copago.
 7. Generación del detalle.
 8. Herencia.
 9. Abstracción.
 10. Polimorfismo.
-

Resumen

Ejercicio	Contexto	Foco principal
1	Banca	Herencia, abstracción y reglas de negocio
2	Salud	Abstracción, sobrescritura y polimorfismo
3	Finanzas	Clases abstractas, interfaces y cálculos
4	Seguros	Herencia, interfaces y polimorfismo
Integrador 1	Banca	Diseño de productos financieros
Integrador 2	Salud	Diseño de prestaciones y cobertura

Consideraciones generales

En todos los ejercicios se debe priorizar que cada clase tenga responsabilidades coherentes con el problema que representa.

La herencia debe utilizarse para representar una verdadera relación de especialización.

Las clases abstractas deben representar conceptos generales que comparten comportamiento y que, al mismo tiempo, requieren implementaciones específicas en las clases concretas.

Las interfaces deben representar capacidades o contratos del dominio.

El polimorfismo debe permitir ejecutar operaciones sobre una abstracción sin depender directamente del tipo concreto del objeto.

El método `main` debe utilizarse principalmente para crear objetos, enviar parámetros, invocar métodos y comprobar las reglas de negocio. La construcción de un menú es opcional y no constituye el objetivo principal de la guía.